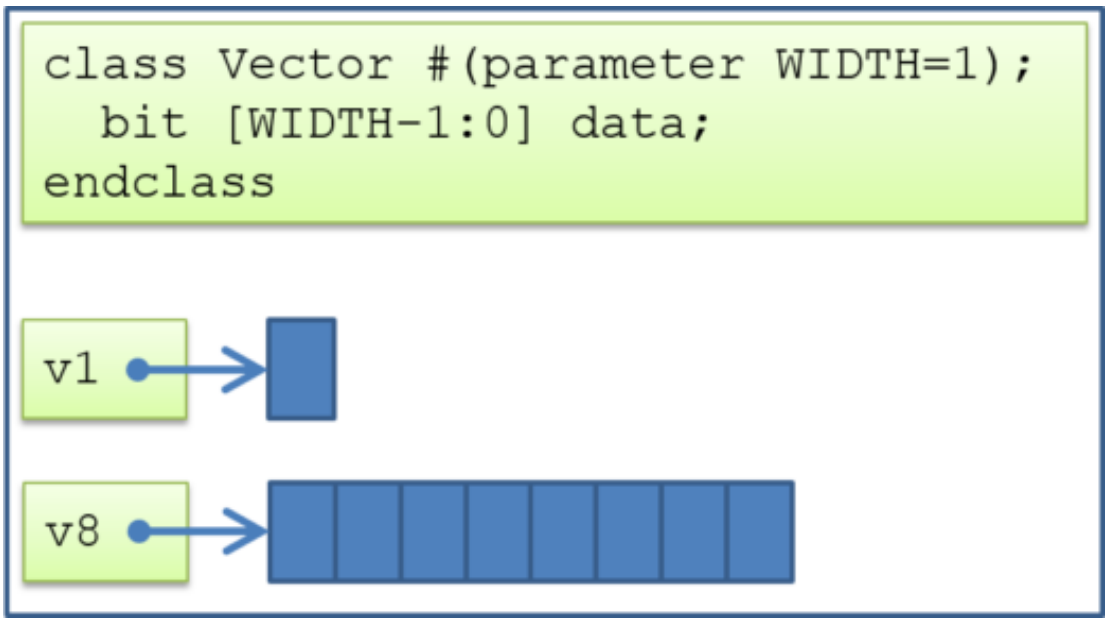
LAB 8: Parameterized Class

Date:25 June of 2025



Proffesor: Dr. Miguel Angel Rivera Acosta

Students: **Luis Alberto Castañeda Montaño**

José Antonio García Madrigal

Cesar Eduardo Inda Ceniceros

Alonso Emmanuel López Macías

Team: 9

Introduction

In this laboratory titled **"Parameterized Class"**, a class was developed in System Verilog using parameters (parameter) to flexibly define the data size (SIZE) and the maximum allowed value (MAX_VALUE) for a random variable (var1). Through the use of parameterized classes, it was possible to instantiate multiple objects of the same class with different configurations, demonstrating how code reuse can remain efficient and scalable within a verification environment.

The class design also includes a **constraint** to limit the value of the random variable, as well as a post_randomize method to display the result of each randomization. Subsequently, a **testbench module** was implemented, in which two objects of the class were created with different parameters, and a randomization sequence was executed for each of them.

The use of **parameterized classes** in System Verilog is essential for creating **versatile, scalable, and reusable verification environments**. They allow a base class to be adapted to multiple scenarios without duplicating code, facilitating the generation of test stimuli for designs with varying data widths, value limits, or other configurations. Additionally, when combined with **randomization mechanisms (randomize)** and **constraints**, they provide a powerful way to automate test generation within modern verification frameworks such as **UVM (Universal Verification Methodology)**.

Understanding and applying parameterized classes enables verification engineers to develop more efficient, focused, and reusable tests, reducing both time and errors in the hardware validation process.

For do a brief representation for parameterized class.

```
// Declare parameterized class
class <name_of_class> #(<parameters>);
class Trans #(addr = 32);

// Override class parameter
<name_of_class>  #(<parameters>) <name_of_inst>;
Trans #(.addr(16)) obj;
```

Figure: 1 Parameterized Class

This a parameterized class which has *size* as the parameter that can be changed during instantiation.

```
// A class is parameterized by #()
// Here, we define a parameter called "size" and gives it
// a default value of 8. The "size" parameter is used to
// define the size of the "out" variable
class something #(int size = 8);
    bit [size-1:0] out;
endclass

module tb;

  // Override default value of 8 with the given values in #()
  something  #(16) sth1;                // pass 16 as "size" to this class object
  something  #(.size (8)) sth2;         // pass 8 as "size" to this class object
  typedef something #(4) td_nibble;     // create an alias for a class with "size" = 4 as "nibble"
  td_nibble nibble;

  initial begin
    // 1. Instantiate class objects
    sth1 = new;
    sth2 = new;
    nibble = new;

    // 2. Print size of "out" variable. $bits() system task will return
    // the number of bits in a given variable
    $display ("sth1.out   = %0d bits", $bits(sth1.out));
    $display ("sth2.out   = %0d bits", $bits(sth2.out));
    $display ("nibble.out = %0d bits", $bits(nibble.out));
  end
endmodule
```

Figure: 2 Testbench Parameterized Class

Requirements

- Compile & run the program in ***"Vivado".***
- Validate that randomness respects the **"MAX VALUE"** parameter.
- Validate that **"SIZE"** of **"Var1"** is correctly limited.
- Validate the multiple independent instances.

Development

We defined a parameterized class this mean that the behavior can be modified also instance the structure for this class. We set 2 parameters **"SIZE"** & **"MAX_VALUE".** In other part we create a **"constructor"** this execute and create the object with the name. A function void that responds from each call from a post randomize value. Printing the value in console with the value var1 to varn with the name for the object.

```
`timescale 1ns / 1ps

// Course: Certificación en diseño de circuitos integrados digitales
// Client: COCYTEN 2024
// Owner: Team9
// Laboratory 8: parameterized_class_sv

// This is a class, which is a user-defined data type
class base_class #(parameter SIZE = 2, MAX_VALUE = 10);

    string name;
  rand bit [SIZE - 1 : 0] var1; // Class property

  constraint var1_max {var1 <= MAX_VALUE;};

  // The new method is the constructor of the class in SystemVerilog
  // The constructor method is used to create an object of this class type
  function new(string name = "");
    // Assign name input argument to name member of base_class
    this.name = name;
  endfunction

  // Inherited post_randomize method
  function void post_randomize();
    $display("the value of var1 for object %s in post_randomize is:\n %d", this.name, var1);
  endfunction

endclass
```

Figure: 3 Code_Part1

In this other section we define the “testbench” section for a handler caller for some different classes, also create object for each base_class_type using the constructor, generate the randomize objects in cycle “repeat” begin in this only 10 cycles or 10 times for do these calls.

```
module tb;

  // Creates a handler for a variable called obj1, of base_class type
  base_class#( 4, 9) obj1;
  base_class#( .SIZE(8), .MAX_VALUE(200)) obj2;

  initial begin
    // Creates an object of base_class type, and links the object obj1 handler using the constructor
    obj1 = new("obj1");
    // Creates an object of base_class type, and links the object obj2 handler using the constructor
    obj2 = new("obj2");

    repeat(10) begin
        // Invokes the default randomization method that is inherited in all classes created in SystemVerilog
        obj1.randomize();
        obj2.randomize();
    end
    // Finishes the simulation
    $finish;

  end

endmodule
```

Figure: 4 Code_Part2

Vivado

For the purpose of this practice, we chose the **ZCU104** in order to generate the simulation and project components through the selection of boards.

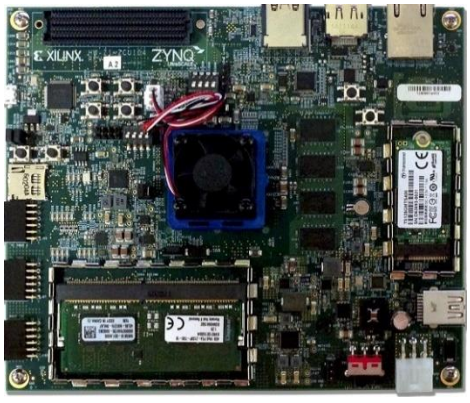


Figure: 4 ZCU104_Board

We create 2 folders one of them:

documentation → for add the information relevant in the laboratory.

rtl → for main design.

dv → for testbench's & simulations

TestBench

In this case, the behavior in the **console** was as follows.

If we run the base code without modification, for the first 3 values, we can see the following:

```
the value of var1 for object obj1 in post_randomize is:
  7
the value of var1 for object obj2 in post_randomize is:
  24
the value of var1 for object obj1 in post_randomize is:
  1
the value of var1 for object obj2 in post_randomize is:
  24
```

Figure: 5 First_Test

If we change this section by **"value = 2"**, we can observe this in the tlc console:

```
// This is a class, which is a user-defined data type
class base_class #(parameter SIZE = 2, MAX_VALUE = 10);

  string name;
 rand bit [SIZE - 2 : 0] var1; // Class property

 constraint var1_max {var1 <= MAX_VALUE;};
```

Figure: 6 Modification1

In this other section we add 2 some code for see the behavioral in the program we tested this part, we set 2 different considerations for each one-bit declaration type value.

For “var1 = rand bit”, for “var2 = protected bit” is a parameter protected. Then we set display for var2 consideration. Also, in the “loop section” we set 3 cycles repeat for each object in this case are 2 object = 6 values in the console.

```
// This is a class, which is a user-defined data type
class base_class #(parameter SIZE = 2, MAX_VALUE = 100);

  string name;
  rand bit      [SIZE - 1 : 0] var1; // Class property
  protected bit [SIZE - 2 : 0] var2; // Class property

  constraint var1_max {var1 <= MAX_VALUE;};

  // The new method is the constructor of the class in SystemVerilog
  // The constructor method is used to create an object of this class type
  function new(string name = "");
    // Assign name input argument to name member of base_class
    this.name = name;
  endfunction

  // Inherited post_randomize method
  function void post_randomize();
    $display("the value of var1 for object %s in post_randomize is:\n %d", this.name, var1);
    $display("the value of var1 for object %s in post_randomize is:\n %d", this.name, var2);
  endfunction

endclass
```

Figure: 7 Modification2

We see this in the console:

```
the value of var1 for object obj1 in post_randomize is:
  7
the value of var1 for object obj1 in post_randomize is:
 0
the value of var1 for object obj2 in post_randomize is:
  24
the value of var1 for object obj2 in post_randomize is:
  0
the value of var1 for object obj1 in post_randomize is:
  1
the value of var1 for object obj1 in post_randomize is:
 0
the value of var1 for object obj2 in post_randomize is:
  24
the value of var1 for object obj2 in post_randomize is:
  0
the value of var1 for object obj1 in post_randomize is:
  3
the value of var1 for object obj1 in post_randomize is:
 0
the value of var1 for object obj2 in post_randomize is:
 152
the value of var1 for object obj2 in post_randomize is:
  0
```

Figure: 8 Console1

If we comment the display line for var2, we see this behavioral:

```
the value of var1 for object obj1 in post_randomize is:
  7
the value of var1 for object obj2 in post_randomize is:
  24
the value of var1 for object obj1 in post_randomize is:
  1
the value of var1 for object obj2 in post_randomize is:
  24
the value of var1 for object obj1 in post_randomize is:
  3
the value of var1 for object obj2 in post_randomize is:
 152
```

Figure: 9 Console2

GitHub Repository

https://github.com/CeicGitHub/Fundamentals_Of_Digital_Design_FPGA_5/tree/Lab8_ParameterizedClass

Conclusion / Issues

José Antonio Garcia Madrigal

In this practice, the size *"(SIZE) and the maximum value (MAX_VALUE)"* of a signal were parameterized — a common approach in the design of modules such as registers, buses, or ALUs that require flexibility in data width. This ability to parameterize reinforces the concepts of abstraction and scalability, which are key elements in the efficient development of complex digital systems.

Alonso Emmanuel Lopez Macias

System Verilog extends Verilog with object-oriented features, and parameterized classes are a fundamental tool in functional verification. In this practice, the use of the randomize () function along with constraints "(constraint)" demonstrates how a wide range of test data can be generated automatically, a crucial capability for verifying that a design behaves correctly under different conditions. This directly contributes to broader coverage and reduces the risk of undetected errors.

Luis Alberto Castañeda Montaño

Modern practices such as encapsulation (protected) and method overriding “(post_randomize)” were used, which are helpful for hiding internal details and customizing behavior after randomization. This approach promotes modularity and code maintainability, both in design and testing environments.

Cesar Eduado Inda Ceniceros

The use of parameterized classes enables the modeling of generic and reusable structures that represent configurable digital behaviors. In industrial environments where methodologies like “UVM” (Universal Verification Methodology) are used, parameterized classes form the foundation for building reusable and configurable testbenches.
This practice introduces fundamental concepts in a simple way, which can later be scaled to more robust verification components such as drivers, monitors, and scoreboards in advanced verification environments.